Linked List

Equivalence Relations, Sparse Matrices & Doubly Linked Lists

Joseph Chuang-Chieh Lin (林莊傑)

Department of Computer Science & Engineering,
National Taiwan Ocean University

Fall 2026



Outline

1. Equivalence Relations
2. Sparse Matrices Revisted
3. Doubly Linked Lists




Outline

1 Equivalence Relations

2 Sparse Matrices Revisted

3 Doubly Linked Lists




Equivalence Relation

A relation over a set S is said to be an equivalence relation over S iff it is symmertric, reflexive, and transitive over S.

- **reflexive**: $x \equiv x$ for each $x \in S$.
- **symmetric**: for $x, y \in S$, if $x \equiv y$, then $y \equiv x$.
- **transitive**: for x, y, z, if $x \equiv y$ and $y \equiv z$, then $x \equiv z$.




Equivalence Relation

A relation over a set S is said to be an equivalence relation over S iff it is symmertric, reflexive, and transitive over S.

- **reflexive**: $x \equiv x$ for each $x \in S$.
- **symmetric**: for $x, y \in S$, if $x \equiv y$, then $y \equiv x$.
- **transitive**: for x, y, z, if $x \equiv y$ and $y \equiv z$, then $x \equiv z$.

Let's see several examples...

- Fix an integer $n > 0$. Two integers $x, y \in \mathbb{Z}$, $x \sim y$ iff $x \equiv y \pmod{n}$.



Equivalence Relation

A relation over a set S is said to be an equivalence relation over S iff it is symmertric, reflexive, and transitive over S.

- **reflexive**: $x \equiv x$ for each $x \in S$.
- **symmetric**: for $x, y \in S$, if $x \equiv y$, then $y \equiv x$.
- **transitive**: for x, y, z, if $x \equiv y$ and $y \equiv z$, then $x \equiv z$.

Let's see several examples...

- Fix an integer $n > 0$. Two integers $x, y \in \mathbb{Z}$, $x \sim y$ iff $x \equiv y \pmod{n}$.
- Given matrices $A, B \in \mathbb{R}^{n \times n}$, $A \sim B$ iff there exists an invertible $P \in \mathbb{R}^{n \times n}$ such that $B = P^{-1}AP$.



Equivalence Relation

A relation over a set S is said to be an equivalence relation over S iff it is symmertric, reflexive, and transitive over S.

- **reflexive**: $x \equiv x$ for each $x \in S$.
- **symmetric**: for $x, y \in S$, if $x \equiv y$, then $y \equiv x$.
- **transitive**: for x, y, z, if $x \equiv y$ and $y \equiv z$, then $x \equiv z$.

Let's see several examples...

- Fix an integer $n > 0$. Two integers $x, y \in \mathbb{Z}$, $x \sim y$ iff $x \equiv y \pmod{n}$.
- Given matrices $A, B \in \mathbb{R}^{n \times n}$, $A \sim B$ iff there exists an invertible $P \in \mathbb{R}^{n \times n}$ such that $B = P^{-1}AP$.
- Given an undirected graph. For any two vertices x, y in the graph, $x \sim y \iff$ Two vertices x and y lie in the same connected component.



Equivalence Relation

A relation over a set S is said to be an equivalence relation over S iff it is symmertric, reflexive, and transitive over S.

- **reflexive**: $x \equiv x$ for each $x \in S$.
- **symmetric**: for $x, y \in S$, if $x \equiv y$, then $y \equiv x$.
- **transitive**: for x, y, z, if $x \equiv y$ and $y \equiv z$, then $x \equiv z$.

Example

Given $0 \equiv 4$, $3 \equiv 1$, $6 \equiv 10$, $8 \equiv 9$, $7 \equiv 4$, $6 \equiv 8$, $3 \equiv 5$, $2 \equiv 11$, $11 \equiv 0$.



Equivalence Relation

A relation over a set S is said to be an equivalence relation over S iff it is symmertric, reflexive, and transitive over S.

- **reflexive**: $x \equiv x$ for each $x \in S$.
- **symmetric**: for $x, y \in S$, if $x \equiv y$, then $y \equiv x$.
- **transitive**: for x, y, z, if $x \equiv y$ and $y \equiv z$, then $x \equiv z$.

Example

Given $0 \equiv 4$, $3 \equiv 1$, $6 \equiv 10$, $8 \equiv 9$, $7 \equiv 4$, $6 \equiv 8$, $3 \equiv 5$, $2 \equiv 11$, $11 \equiv 0$.
We have three equivalent classes:

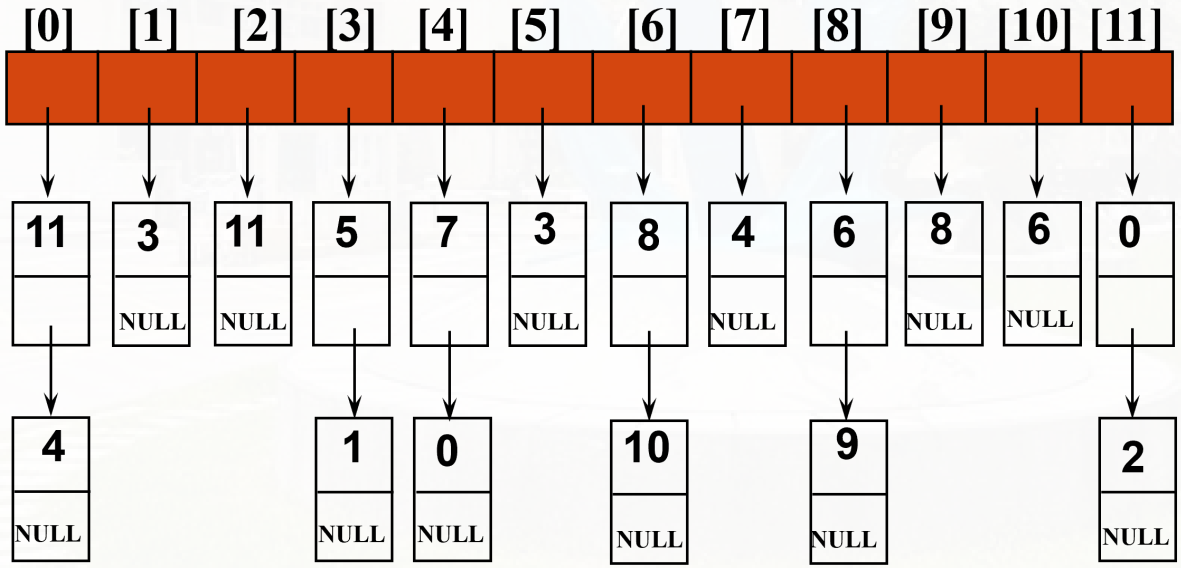
$$\{0, 2, 4, 7, 11\}, \{1, 3, 5\}, \{6, 8, 9, 10\}.$$




Lists after Giving Pairs as the Input

$0 \equiv 4, 3 \equiv 1, 6 \equiv 10, 8 \equiv 9, 7 \equiv 4,$
$6 \equiv 8, 3 \equiv 5, 2 \equiv 11, 11 \equiv 0.$

```
typedef struct node *nodePointer;
typedef struct node {
    int data;
    nodePointer link;
};
```



The Equivalence Algorithm (Full Code HERE)

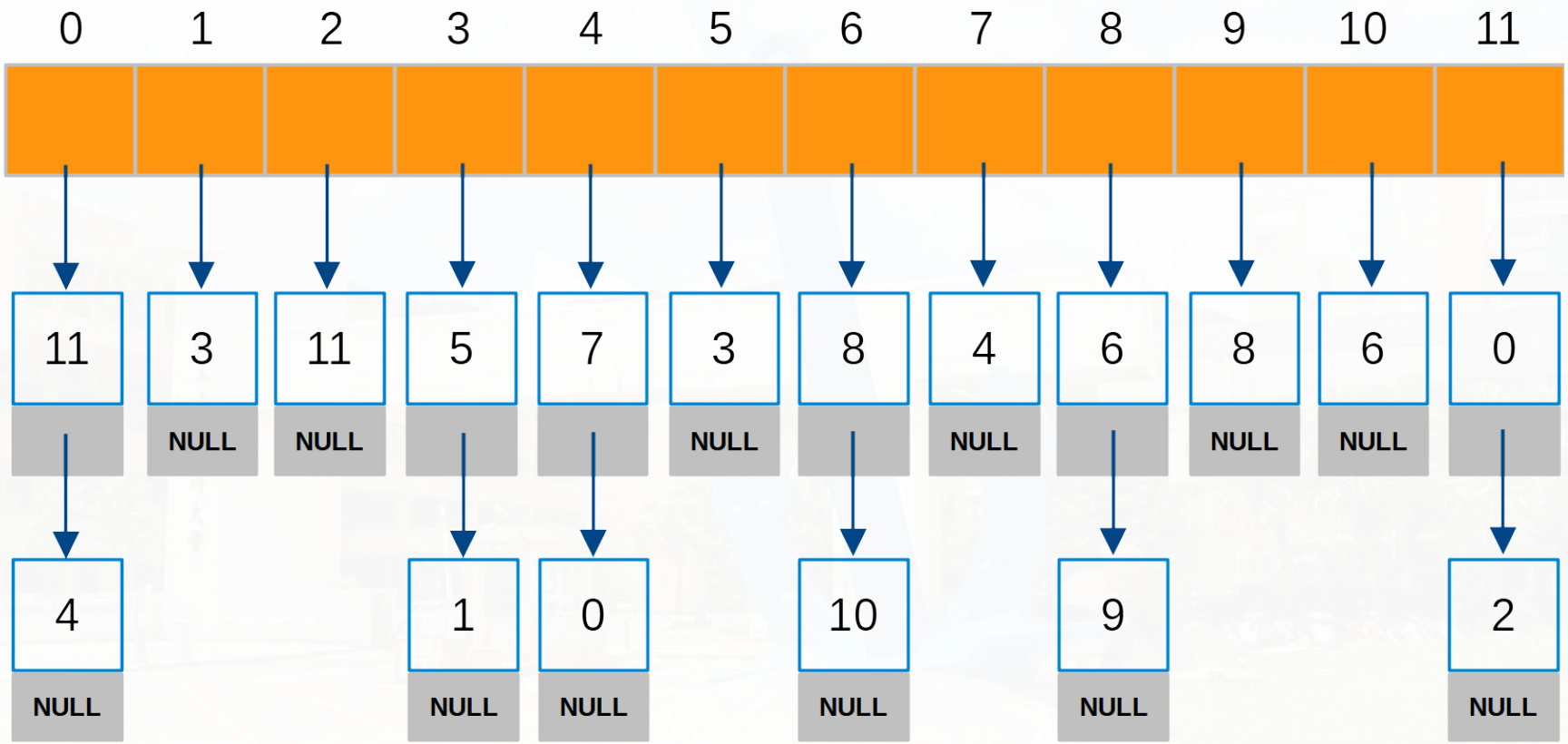
`seq[n]`: hold the head nodes of the n lists; `out[n]`: whether the object i has been printed.

```
void equivalence() {
    initialize seq[i] = NULL, out[i] to true for each i
    while (there are more pairs) {
        read the next pair <i,j>;
        put j on the seq[i] list;
        put i on the seq[j] list;
    }
    for (i=0; i<n; i++) { // the 2nd phase
        if (out[i]) {
            out[i] = false;
            output this equivalence class;
        }
    }
}
```

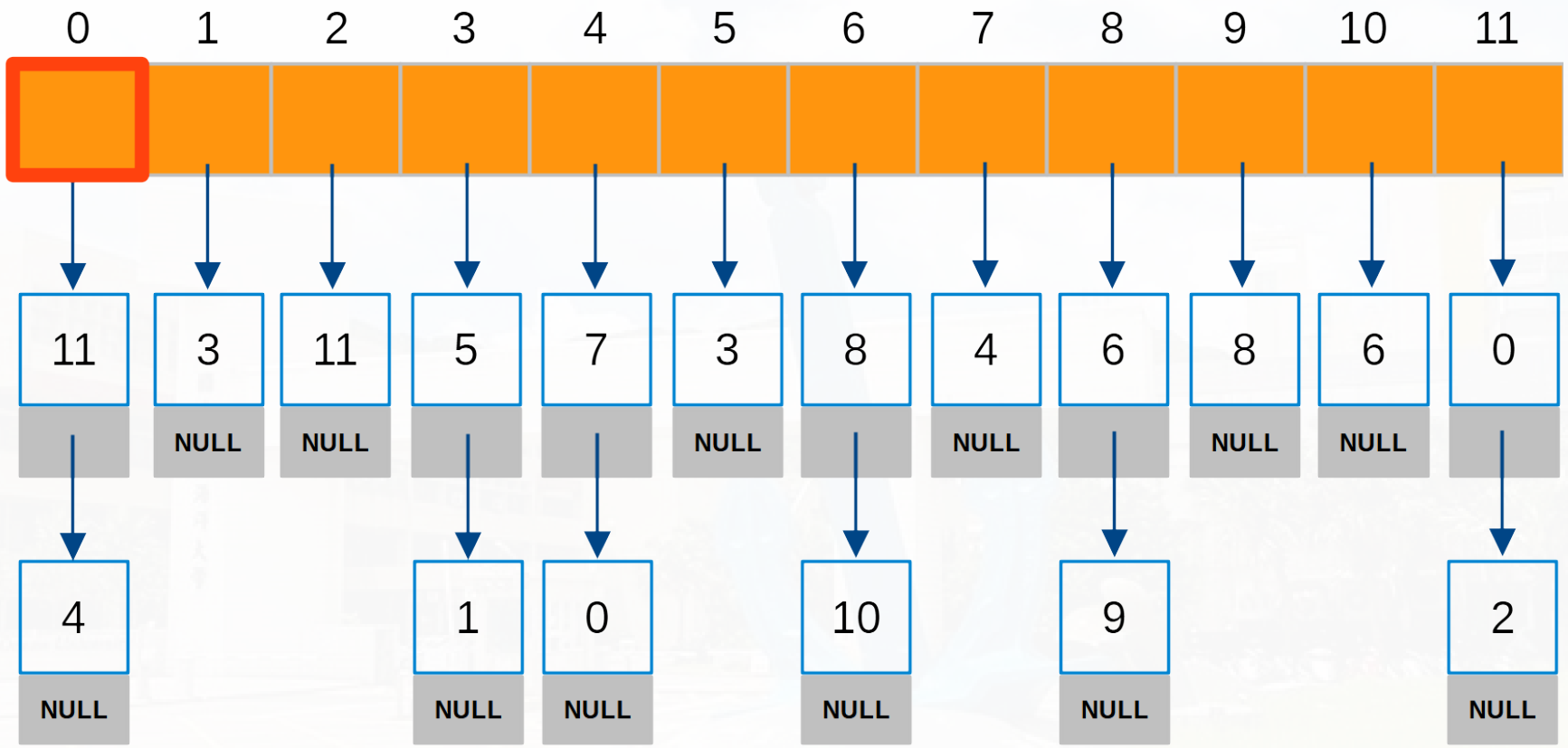



The Equivalence Algorithm (second phase in detail)

```
for (i=0; i<n; i++) { // the 2nd phase
    if (out[i]) {
        printf("\nNew Classes: %d", i);
        out[i] = false; // class involving i must be processed
        x = seq[i]; top = NULL; // initial stack
        while (1) { // find the rest of the class
            while (x) { // process the list
                j = x->data;
                if (out[j]) {
                    printf("%d", j);  out[j] = false;
                    // then, push y into the stack
                    y = x->link; x->link = top; top = x; x = y;
                } else x = x->link;
            }
            if (!top) break;
            // else, the stack keeping equivalent class members is nonempty:
            x = seq[top->data]; top = top->link; // pop the stack
        }
    }
}
```

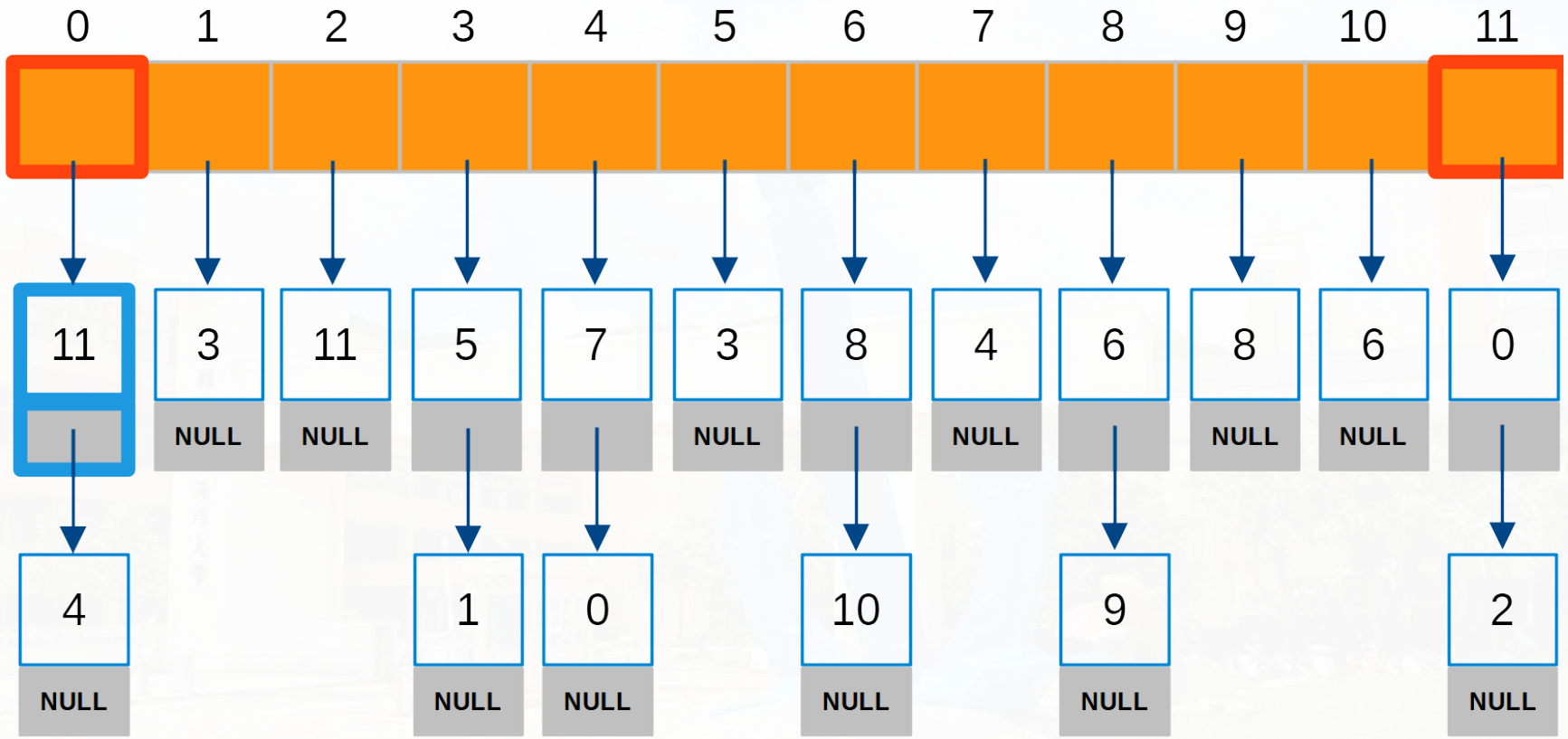


$\text{top} \rightarrow \texttt{NULL}$



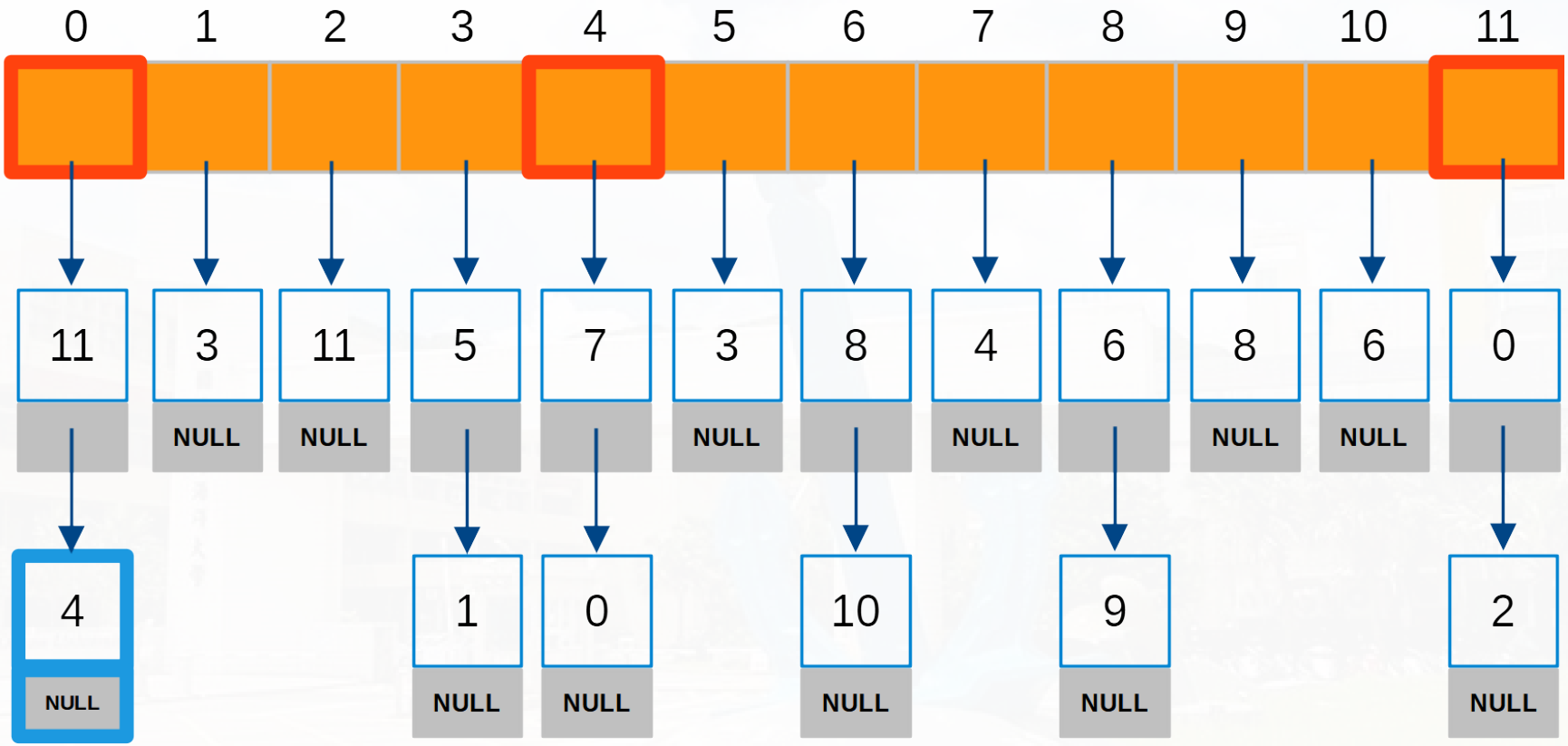
$\text{top} \rightarrow \texttt{NULL}$

New Classes: 0



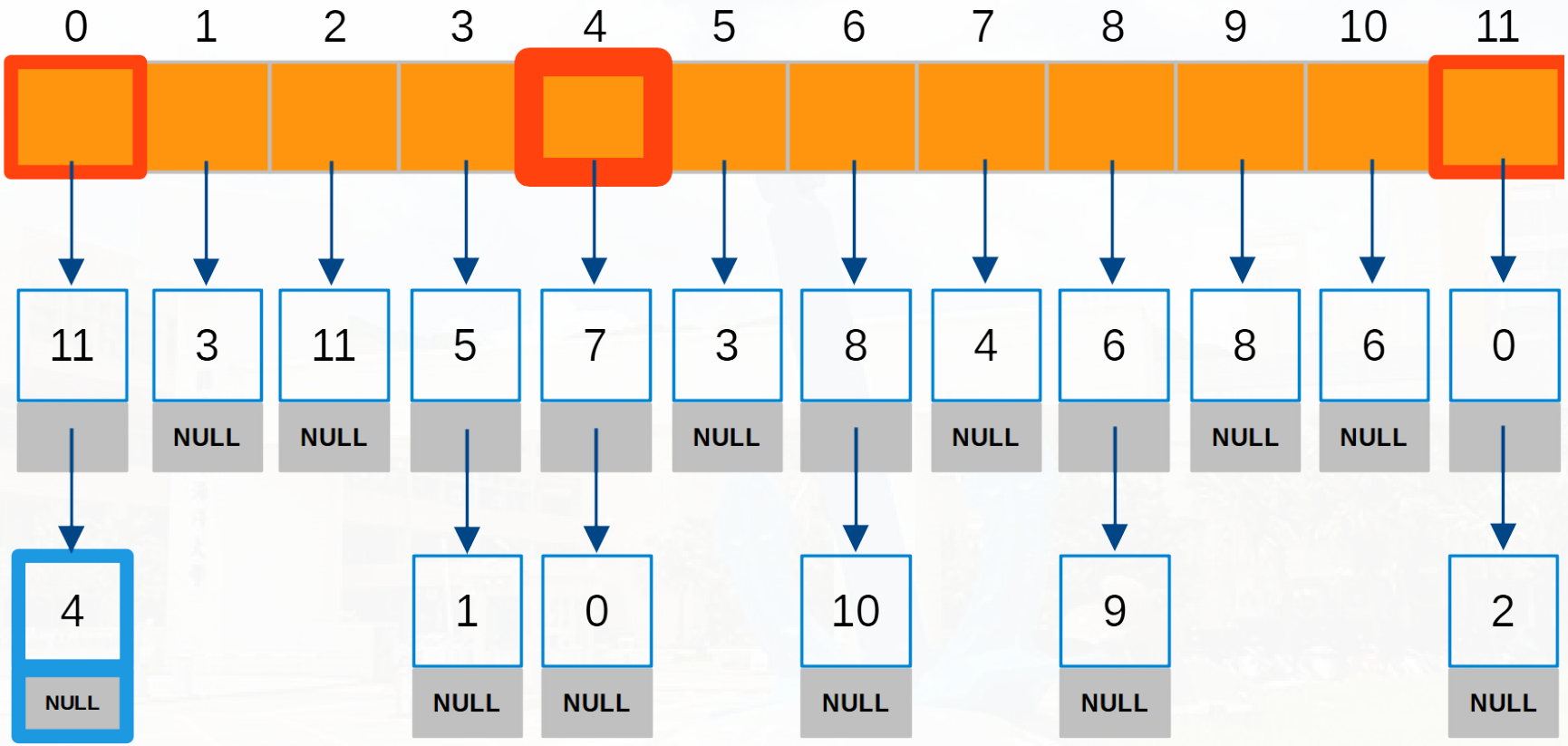
$\text{top} \rightarrow (11) \rightarrow \texttt{NULL}$

New Classes: 0, 11



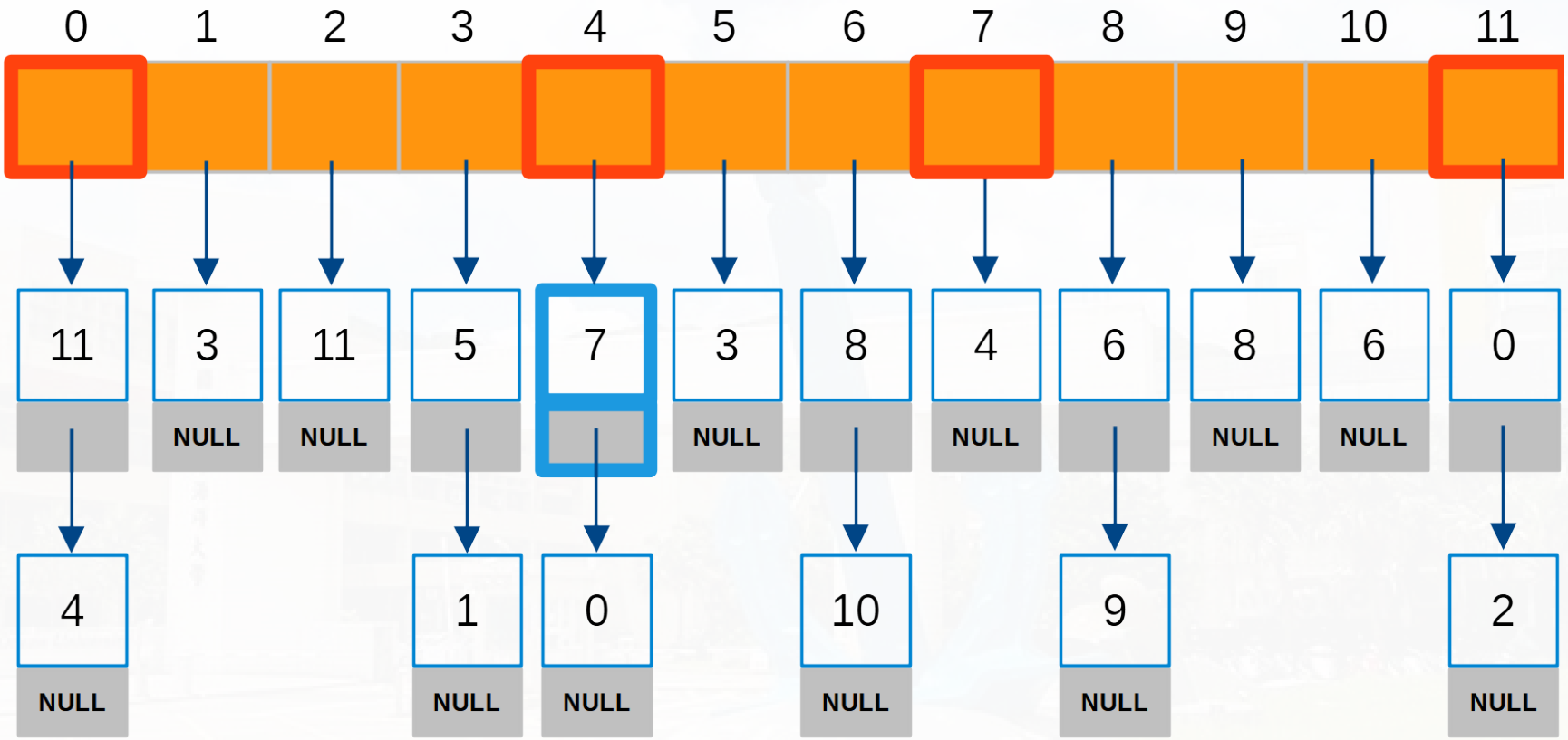
$\text{top} \to (4) \to (11) \to \texttt{NULL}$

New Classes: 0, 11, 4



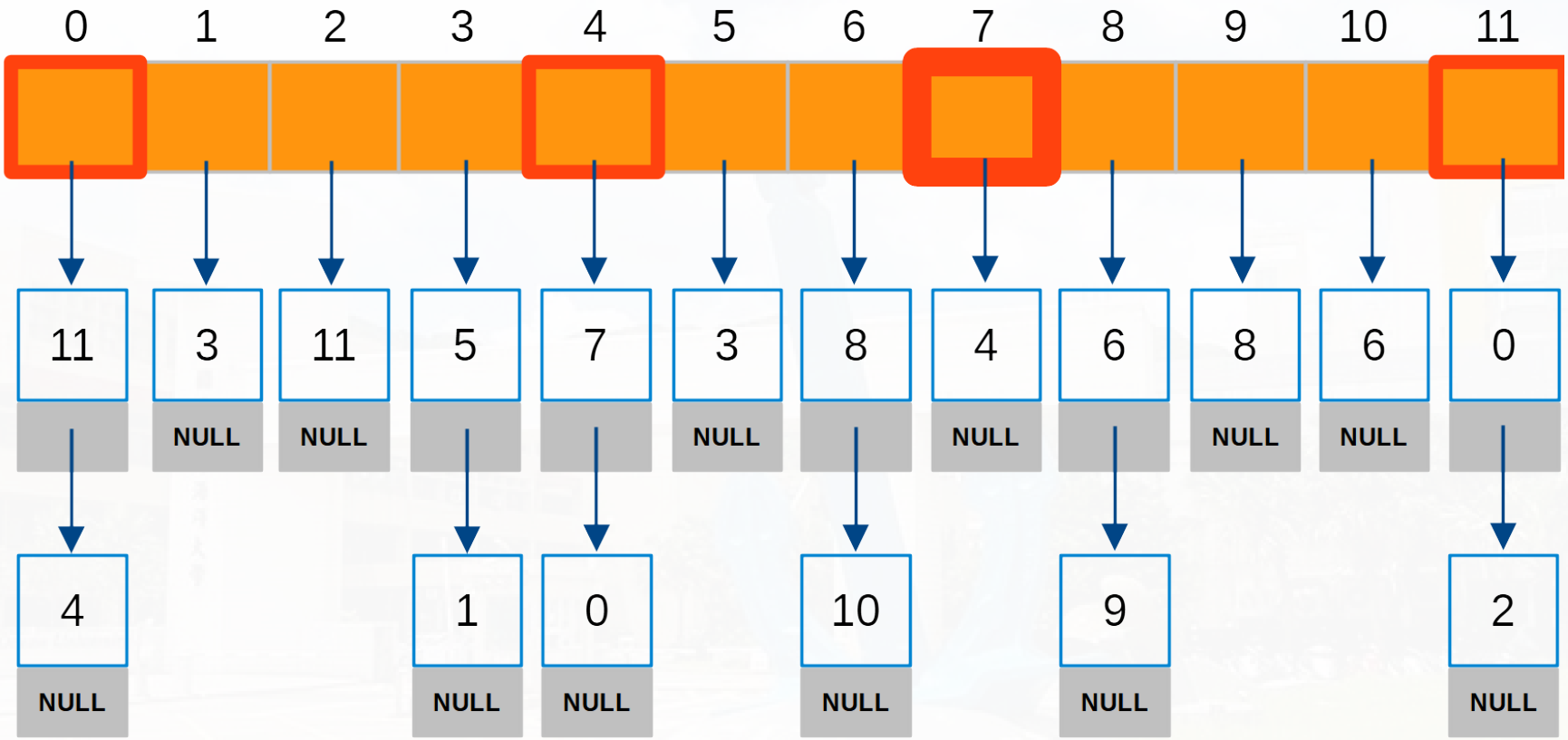
$\text{top} \rightarrow (11) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4



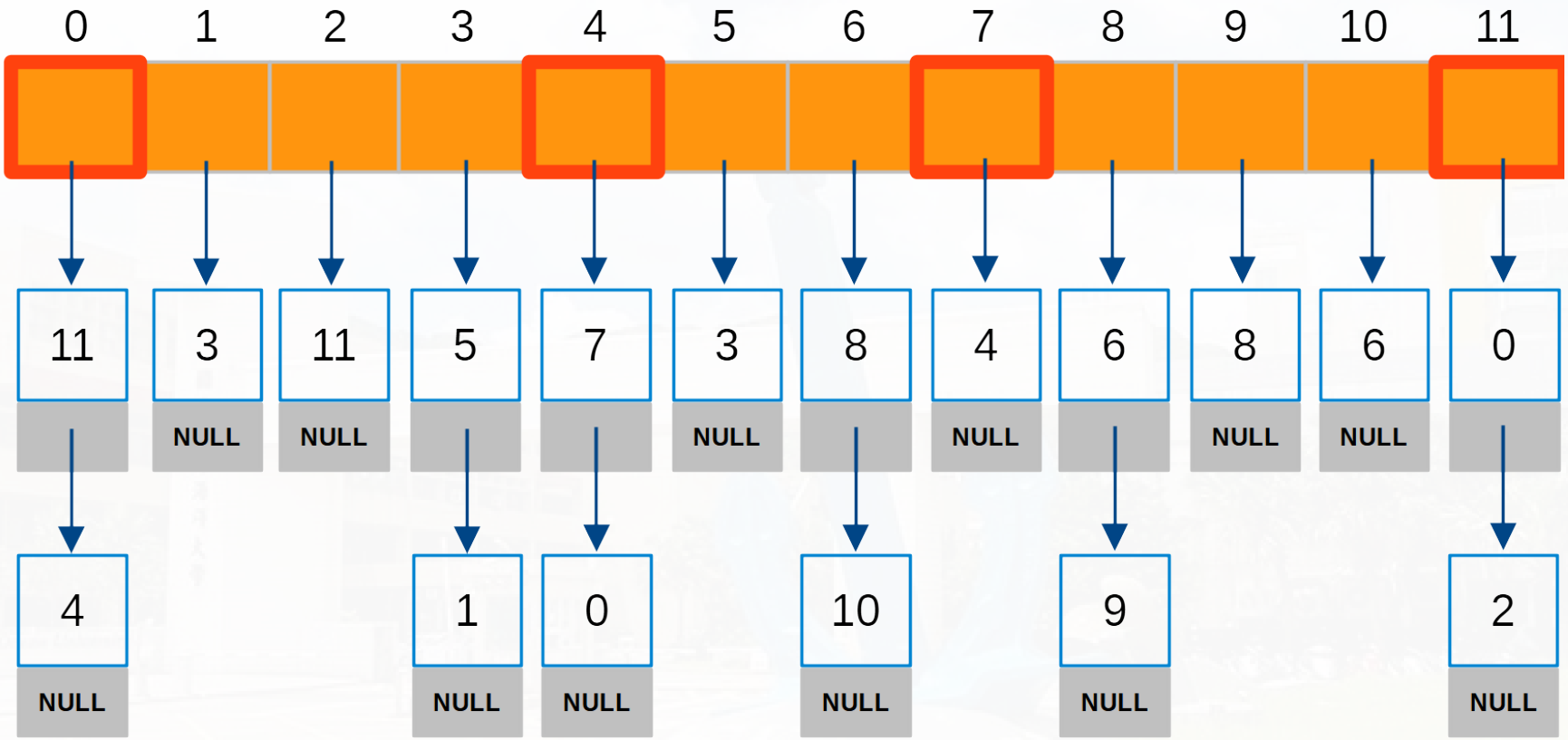
$\text{top} \to (7) \to (4) \to (11) \to \texttt{NULL}$

New Classes: 0, 11, 4, 7



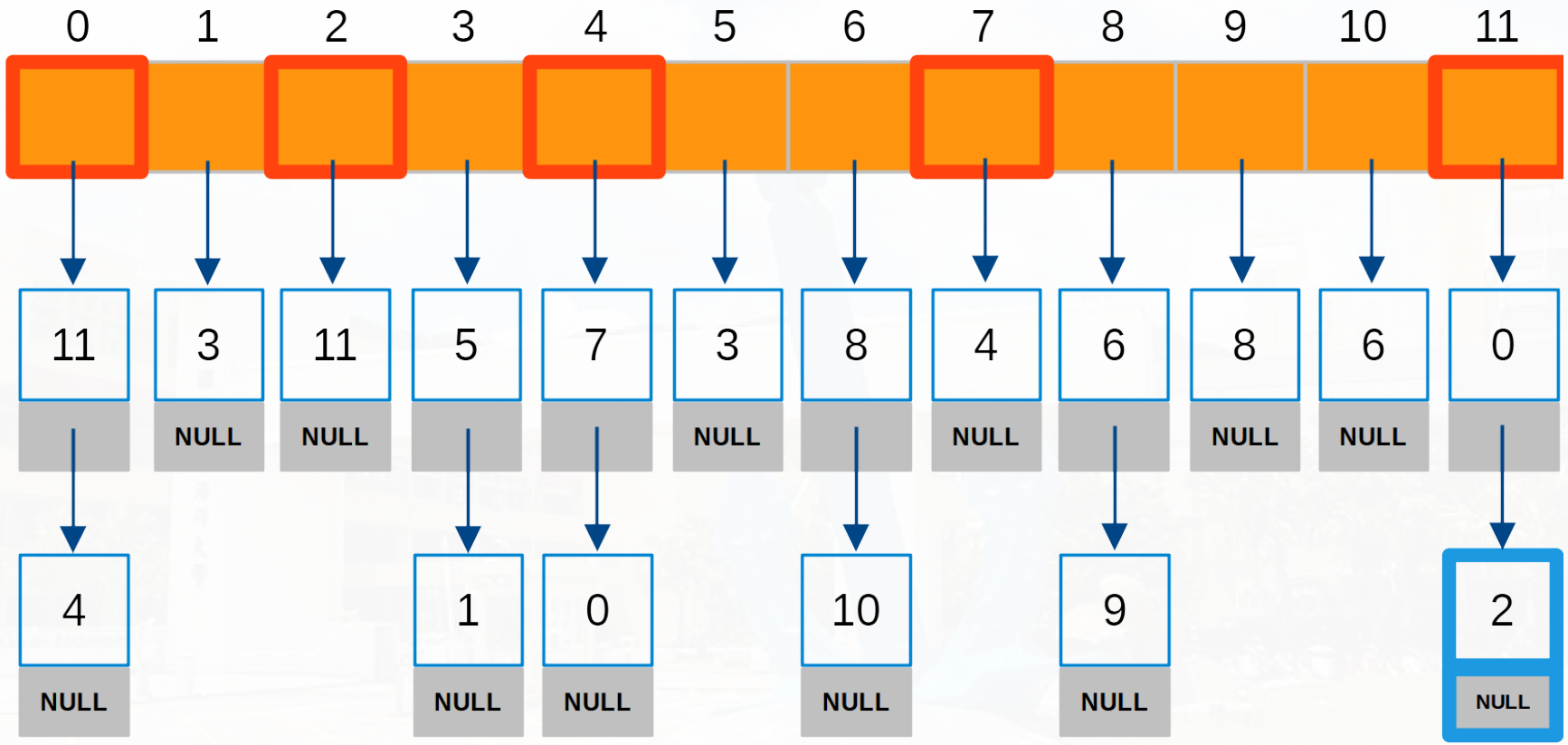
$\text{top} \rightarrow (4) \rightarrow (11) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7



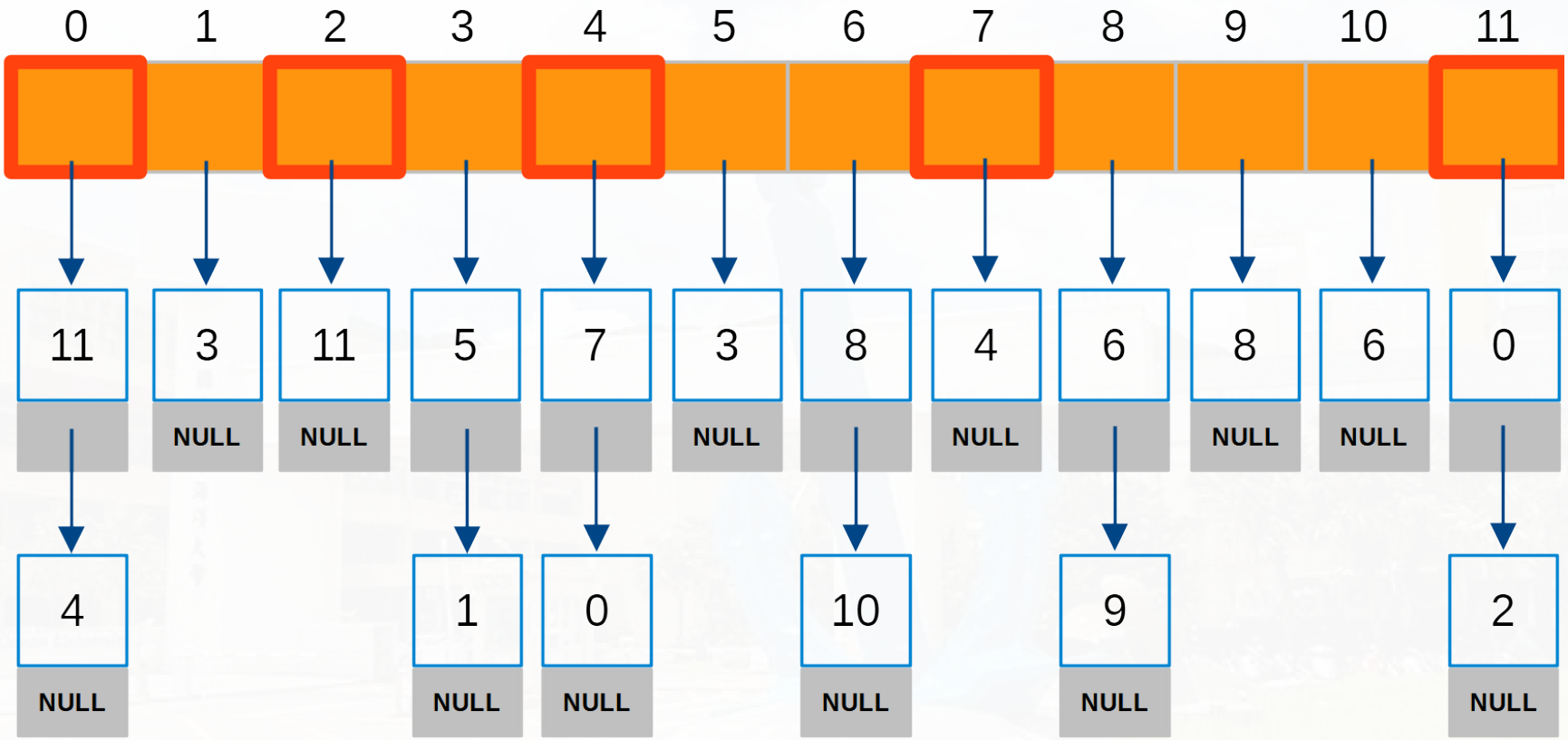
$\text{top} \rightarrow (11) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7



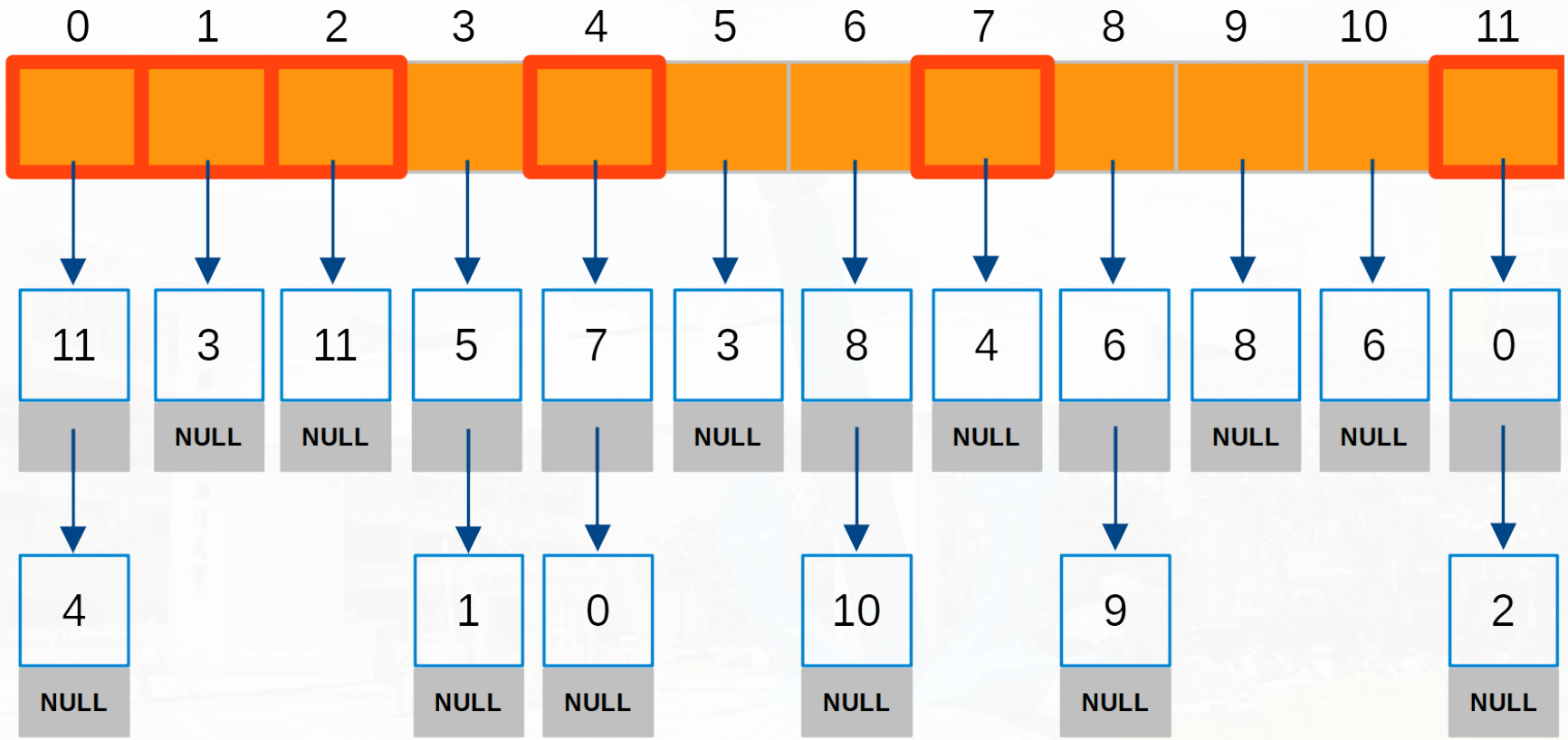
$\text{top} \rightarrow (2) \rightarrow (11) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2



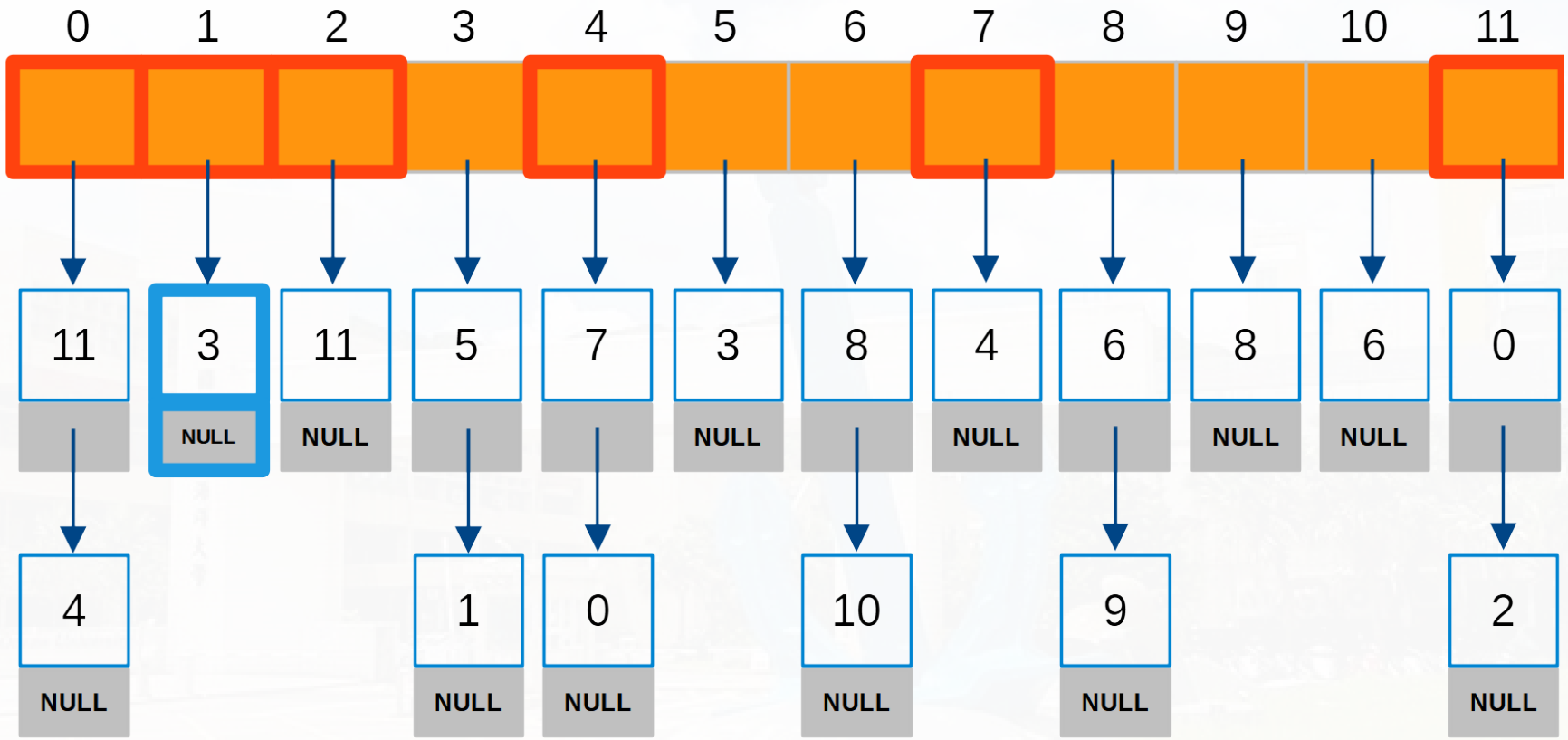
$\text{top} \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2



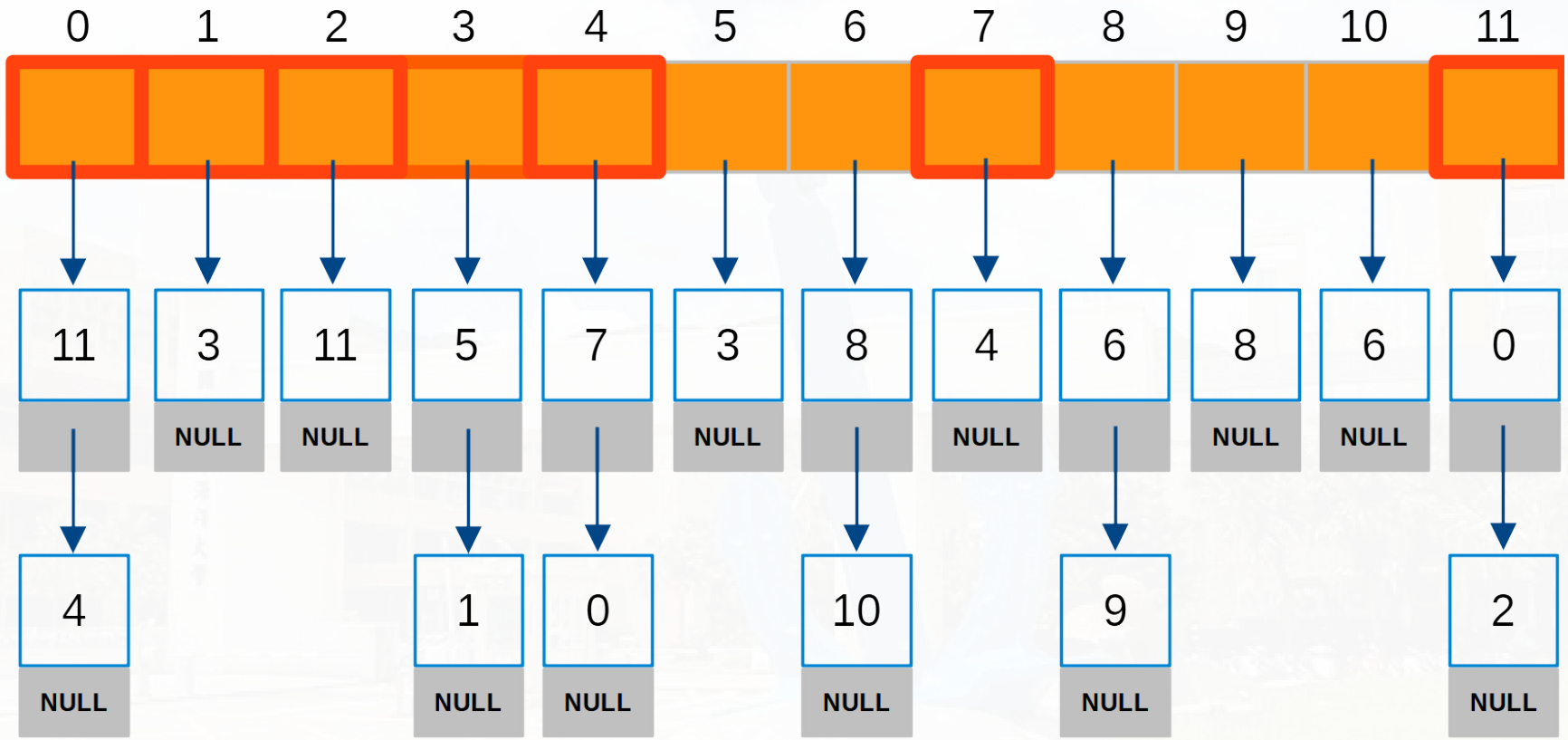
$\text{top} \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1



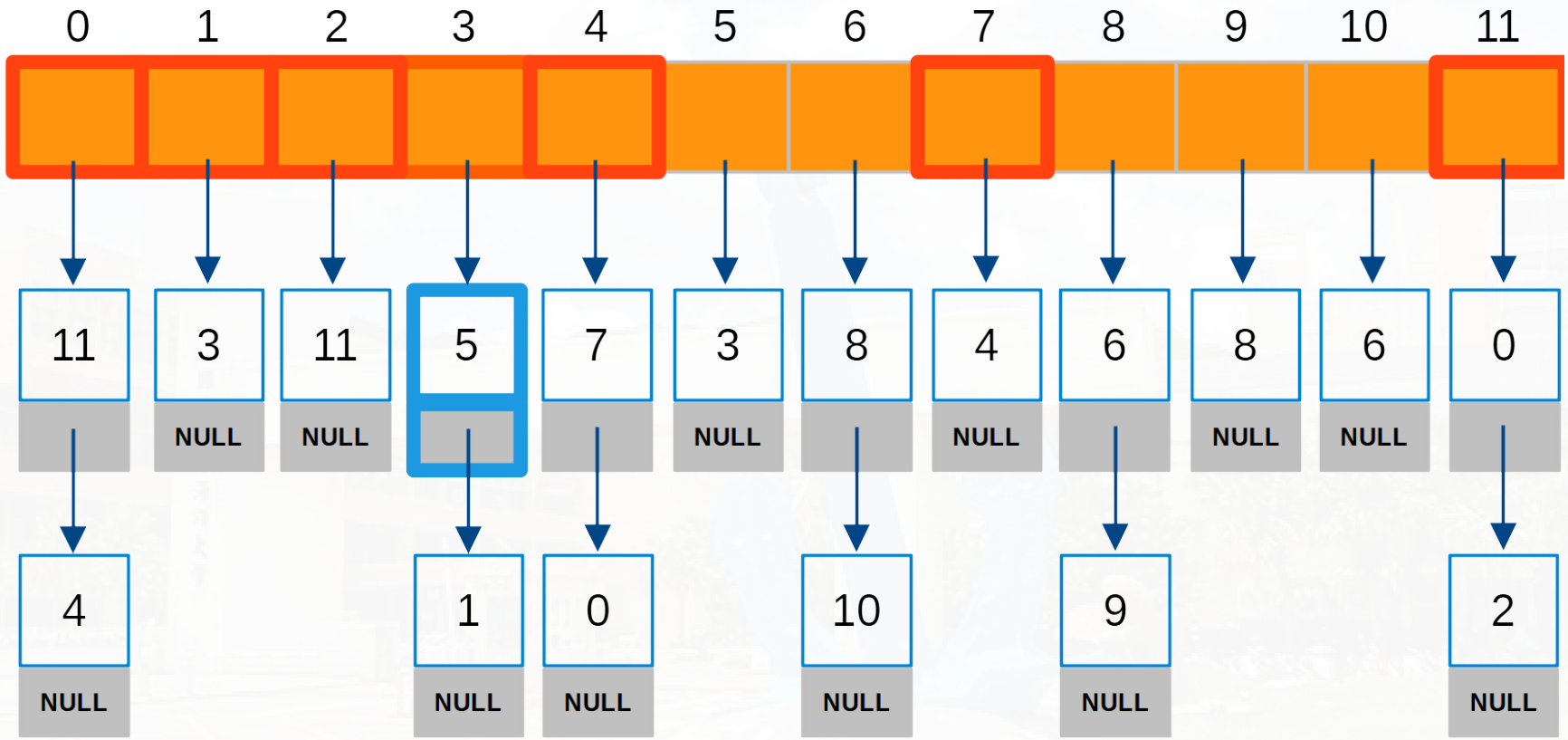
$\text{top} \rightarrow (3) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3



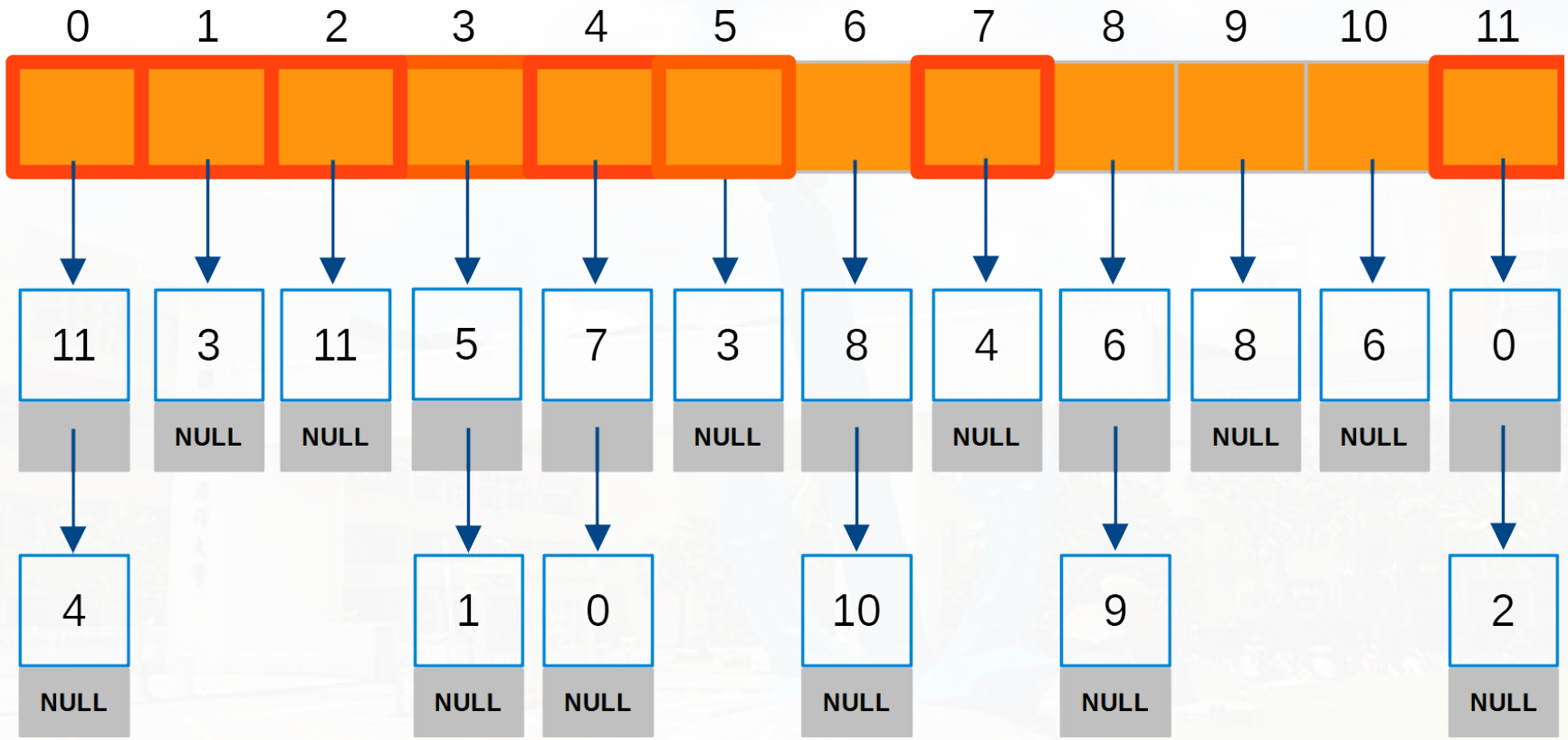
$\text{top} \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3



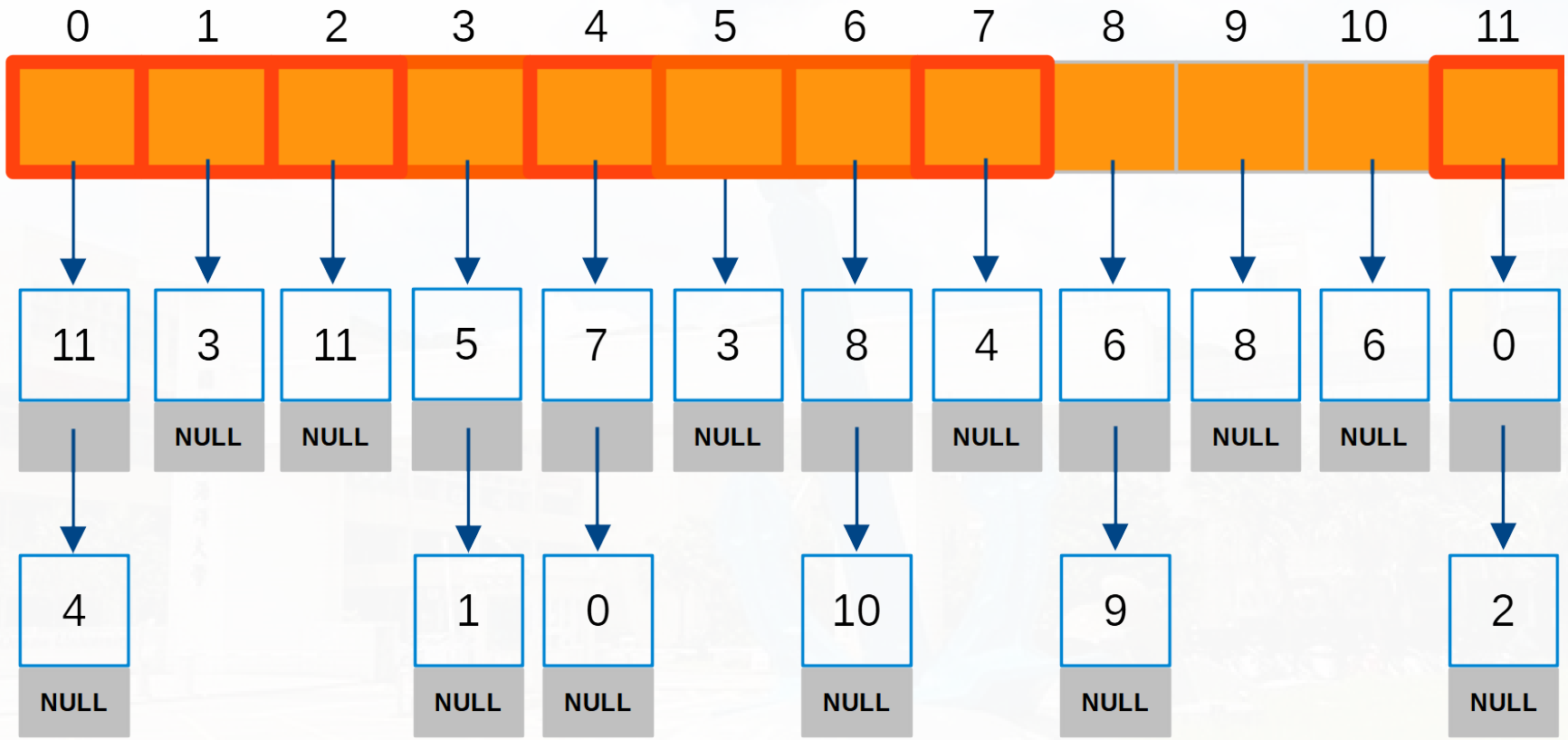
$\text{top} \rightarrow (5) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3, 5



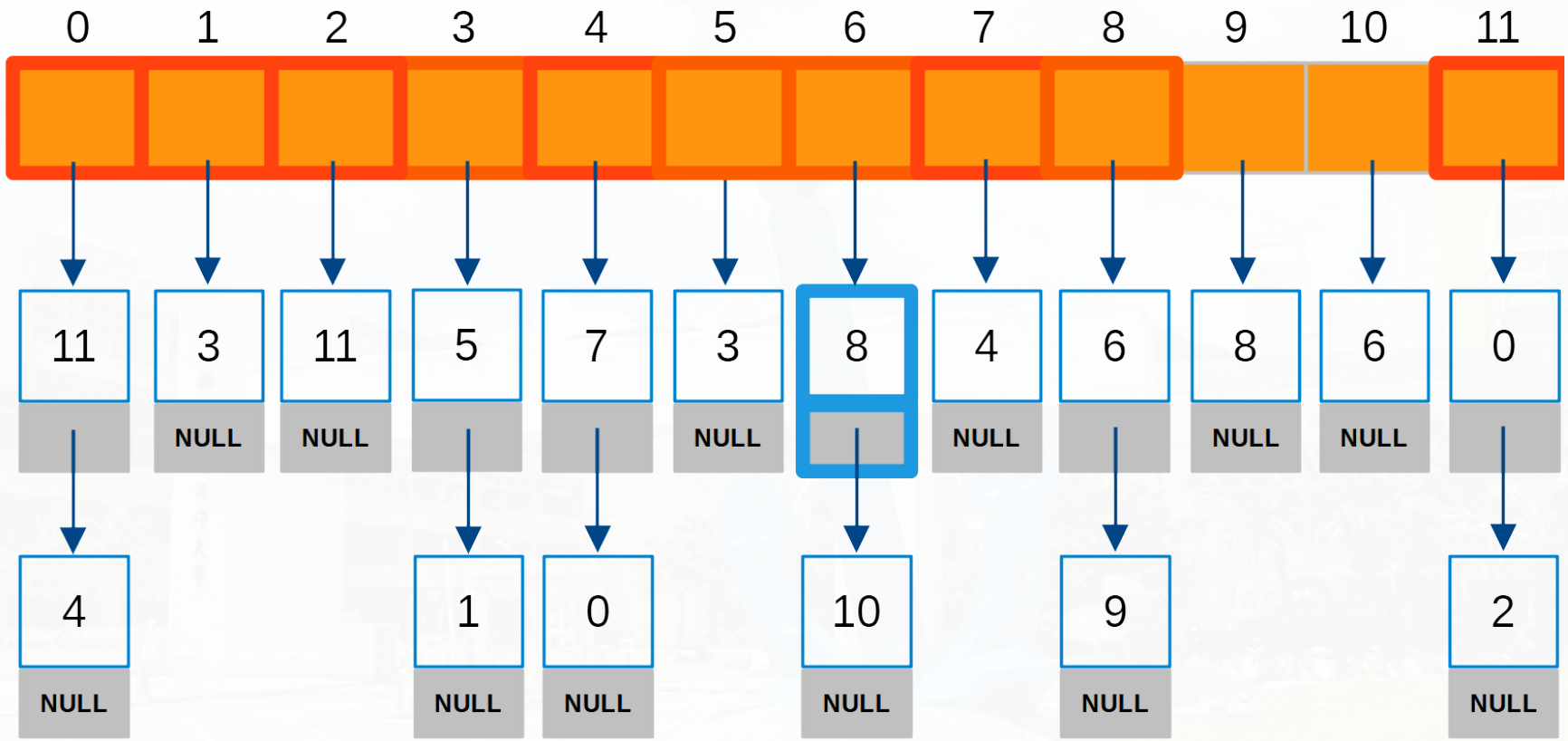
$\text{top} \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3, 5



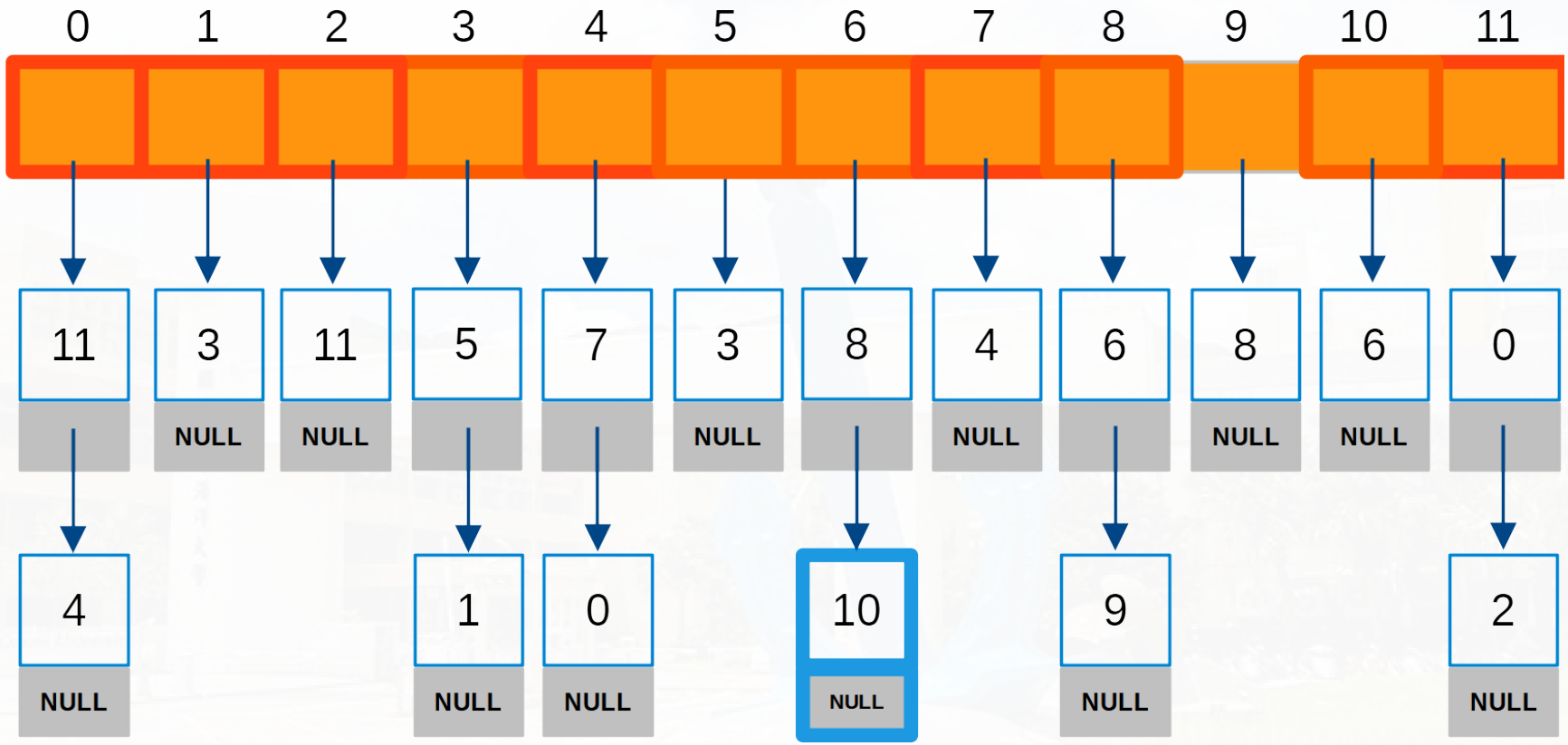
$\text{top} \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3, 5 New Classes: 6



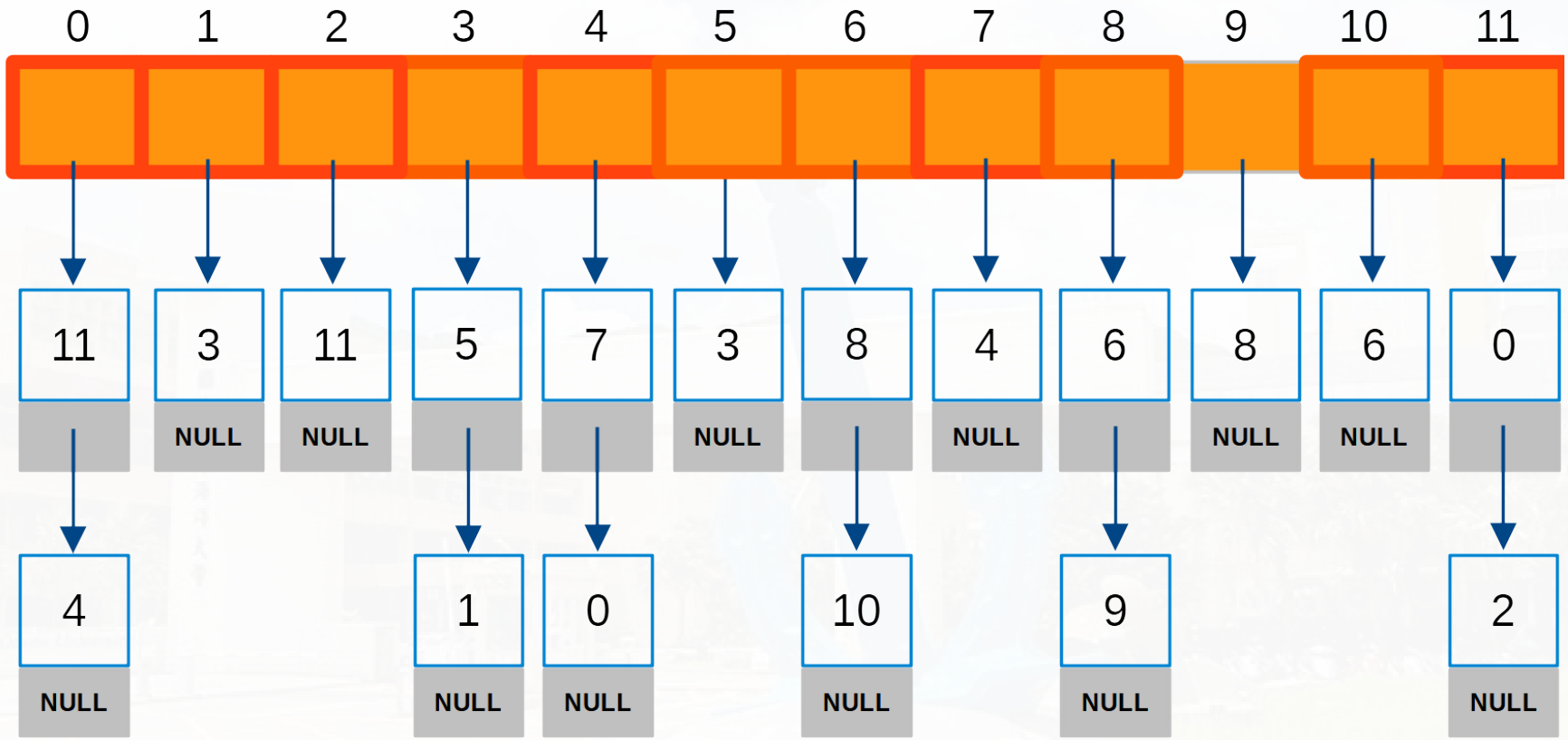
$\text{top} \rightarrow (8) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3, 5 New Classes: 6, 8



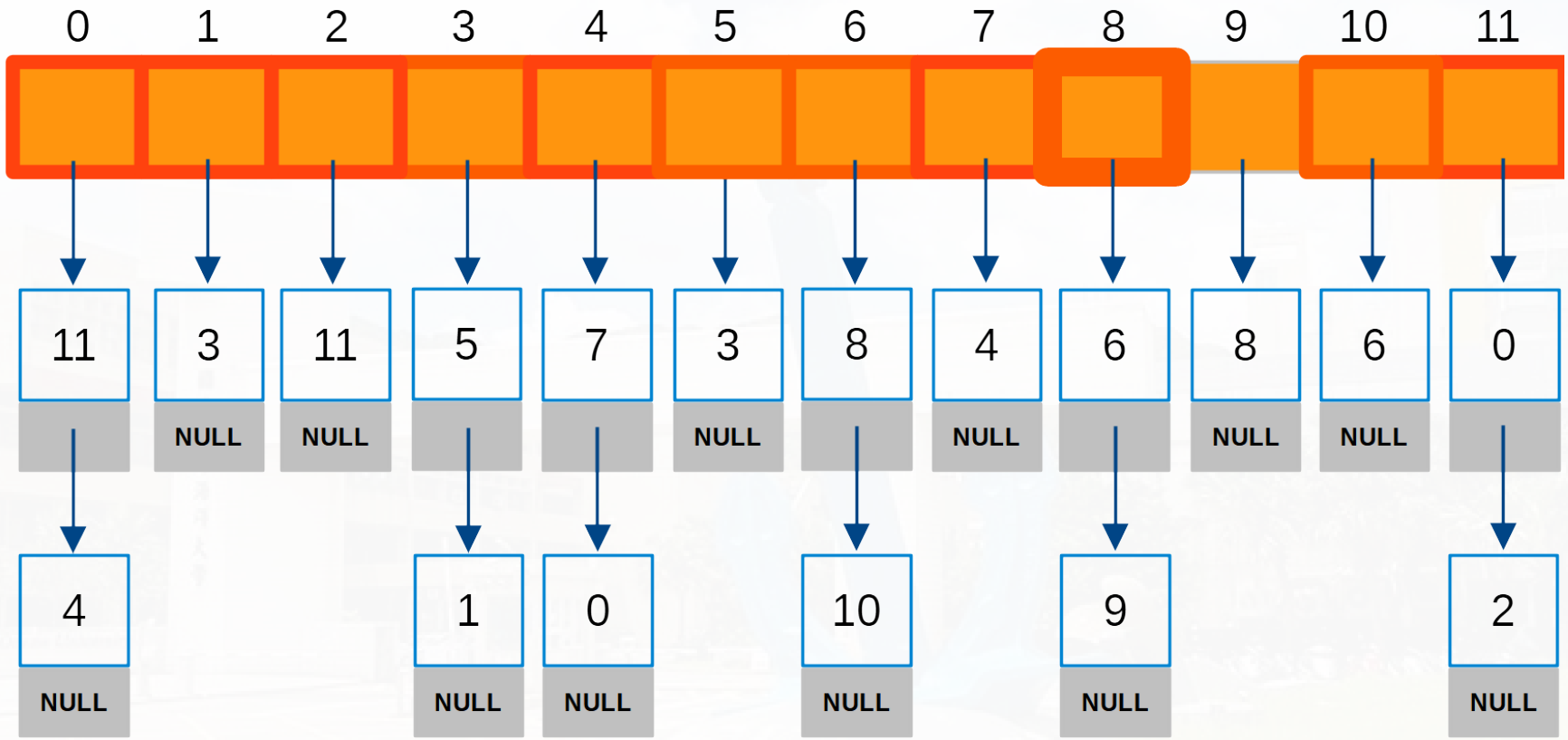
$\text{top} \rightarrow (10) \rightarrow (8) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3, 5 New Classes: 6, 8, 10



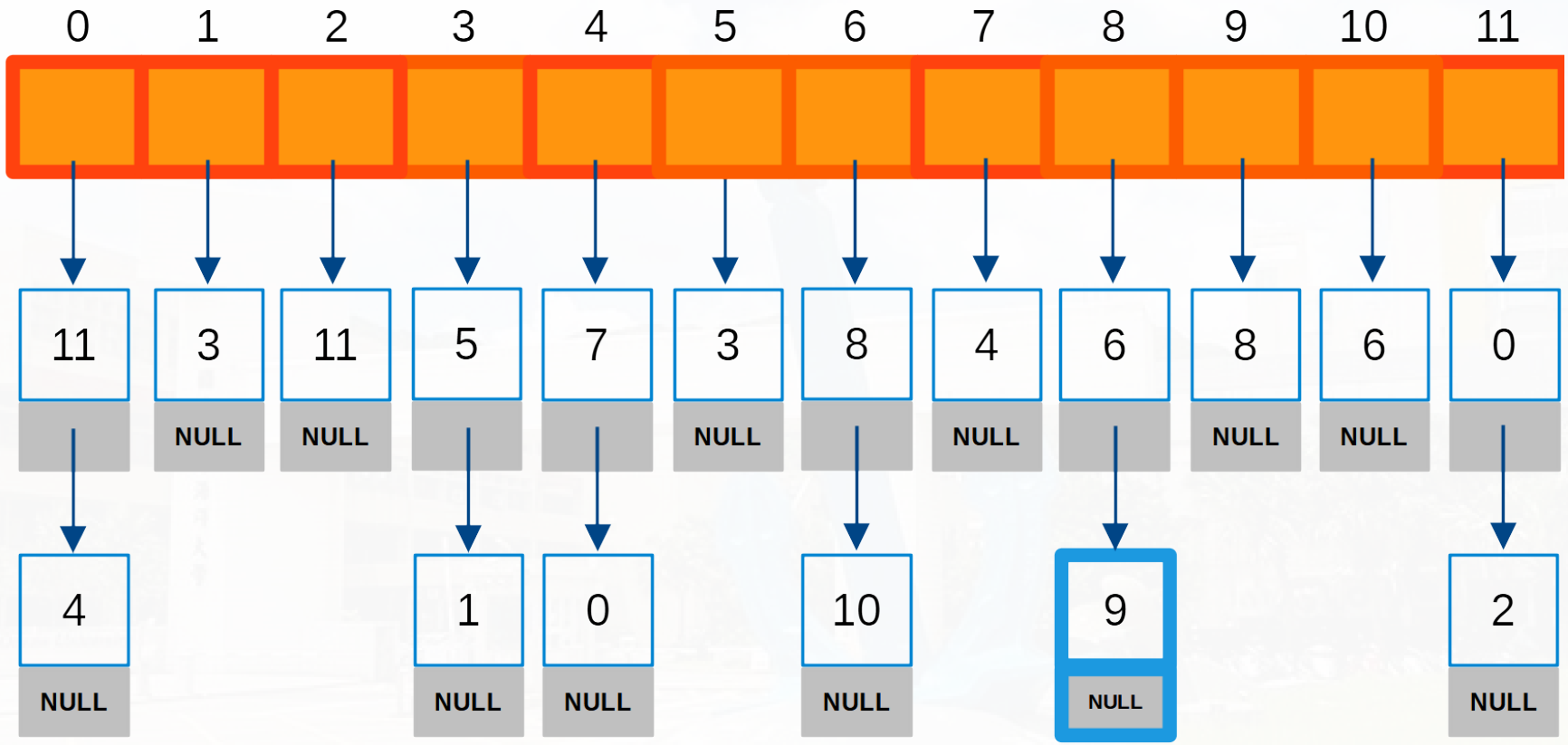
$\text{top} \rightarrow (8) \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3, 5 New Classes: 6, 8, 10



$\text{top} \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3, 5 New Classes: 6, 8, 10



$\text{top} \rightarrow \texttt{NULL}$

New Classes: 0, 11, 4, 7, 2 New Classes: 1, 3, 5 New Classes: 6, 8, 10, 9

Outline

1 Equivalence Relations

2 Sparse Matrices Revisted

3 Doubly Linked Lists



Issues for Previous Array Representation

- When we performed matrix operations such as $+$, $-$, or $*$, the number of **nonzero terms** varied.
- The sequential representation of sparse matrices suffered from the same inadequacies as the similar representation of polynomials.

Solution:

- Linked list representation for sparse matrices.

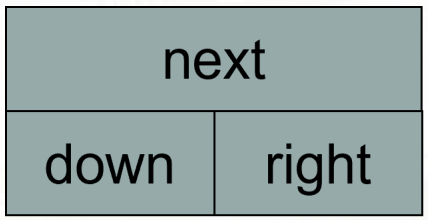


Issues for Previous Array Representation

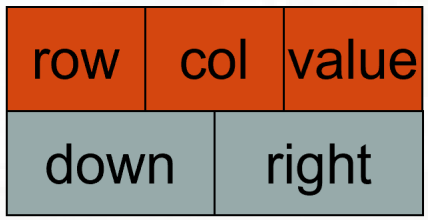
- When we performed matrix operations such as $+$, $-$, or $*$, the number of **nonzero terms** varied.
- The sequential representation of sparse matrices suffered from the same inadequacies as the similar representation of polynomials.

Solution:

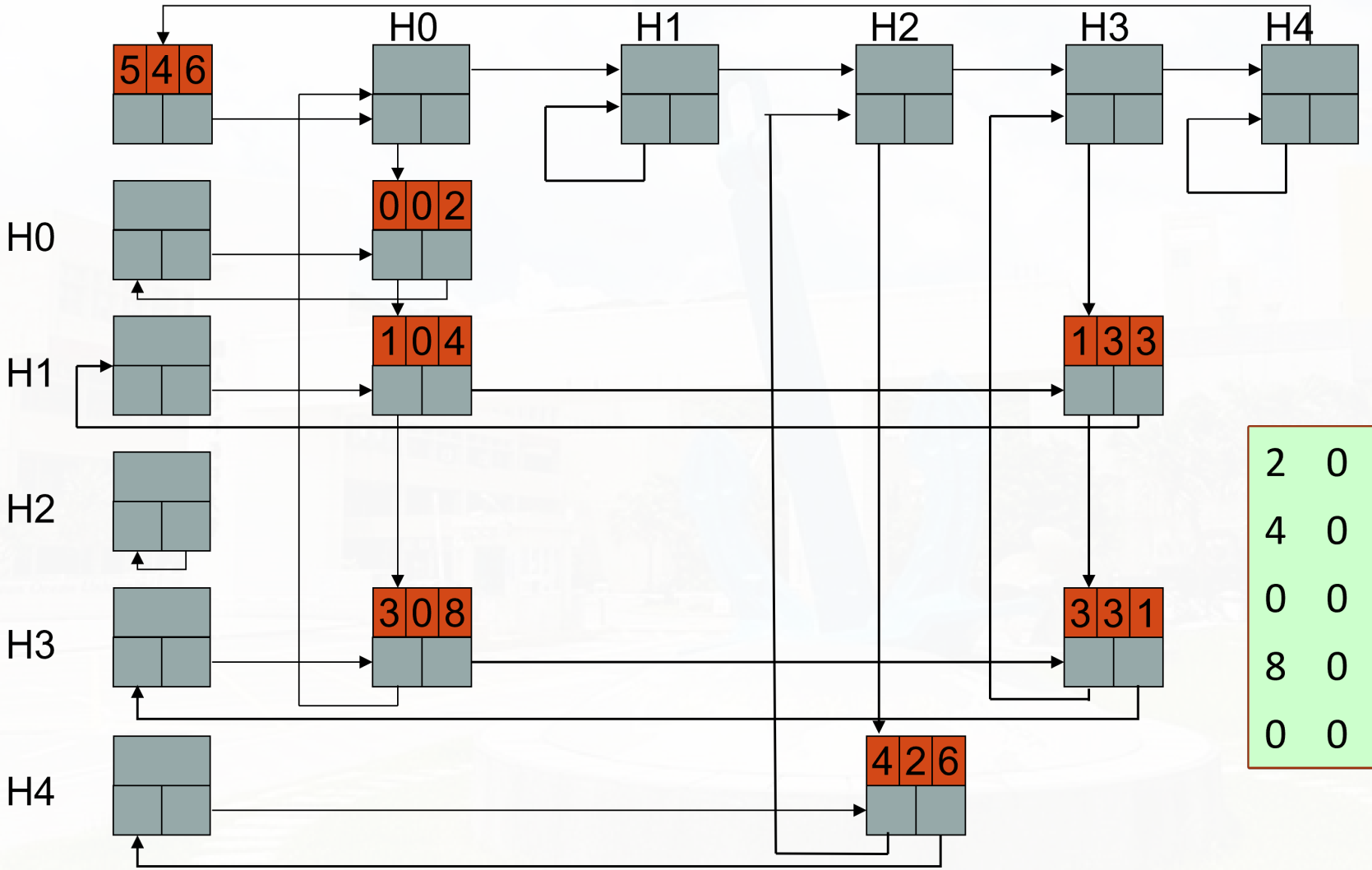
- Linked list representation for sparse matrices.
- Two types of nodes in the representation: header nodes and element nodes.



header node



element node



2	0	0	0
4	0	0	3
0	0	0	0
8	0	0	1
0	0	6	0

Sparse Matrix Representation

- We represent each column (row) of a sparse matrix as a circularly linked list with a header node.
- The header node for row i is also the header node for column i. The number of header nodes is $\max\{\text{numRows}, \text{numCols}\}$.
- Each element node is simultaneously linked into two lists: a row list, and a column list.
- Each head node is belonged to three lists: a row list, a column list, and a header node list.



Outline

1 Equivalence Relations

2 Sparse Matrices Revisted

3 Doubly Linked Lists



Issues for Singly Linked Lists

- The only way to find the node that precedes some node p is to start at the beginning of the list.
- Sometimes it is necessary to move in either direction.

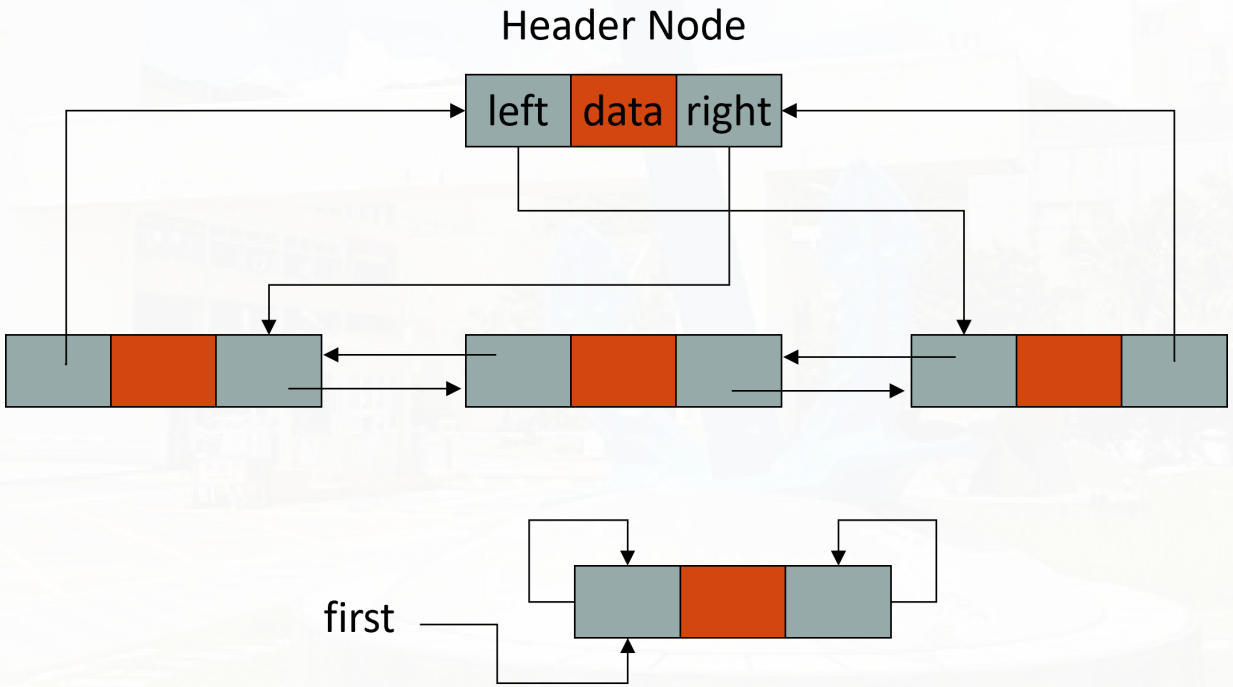
Doubly linked lists:

```
typedef struct node *nodePointer;
typedef struct node {
    nodePointer llink;
    element data;
    nodePointer rlink;
};
```




- The following formula reflects the essential virtue of this structure, namely, that we can go back and forth with equal ease.

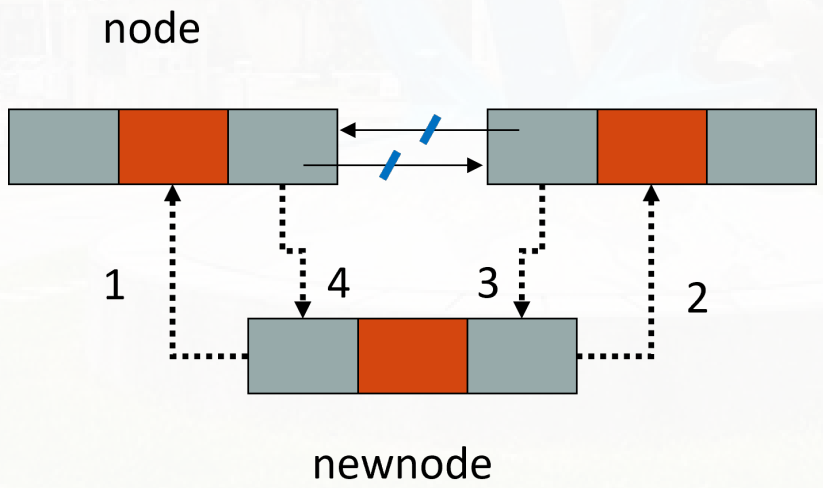
```
ptr = ptr->llink->rlink = ptr->rlink->llink
```



Empty doubly linked circular list with header node

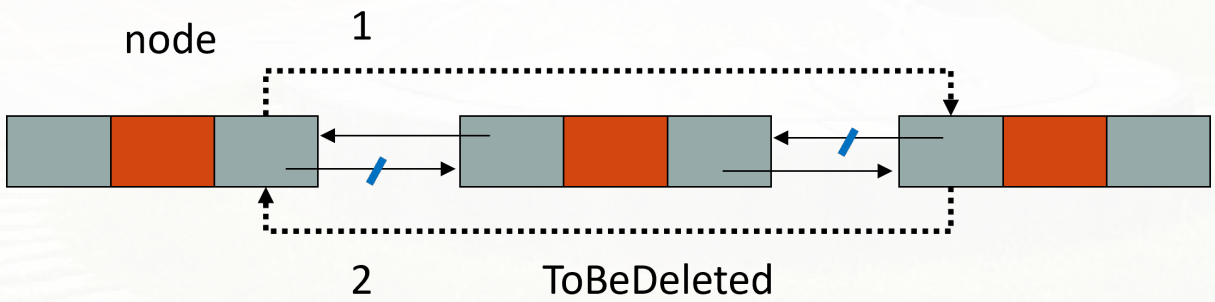
Insertion into a doubly linked circular List

```
void d_LCL_insert(nodePointer node, nodePointer newnode) {
/* insert newnode to the right of node */
    newnode->llink = node;              // 1
    newnode->rlink = node->rlink;       // 2
    node->rlink->llink = newnode;       // 3
    node->rlink = newnode;              // 4
}
// Question: Any other way to do this insertion?
```



Deletion from a doubly linked circular list

```
void d_LCL_delete(nodePointer node, nodePointer ToBeDeleted) {
/* delete from the doubly linked list */
    if (node == ToBeDeleted)
        printf("Deletion of preceding node is not permitted.\n");
    else {
        ToBeDeleted->llink->rlink = ToBeDeleted->rlink;  // 1
        ToBeDeleted->rlink->llink = ToBeDeleted->llink;  // 2
        // Question: Could we switch 1 and 2?
        free(ToBeDeleted);
    }
}
```



Benefit of using the node "ToBeDeleted"

- The node `ToBeDeleted` can be used as a guide for its right node and left node to update the links.
- No additional temporary nodePointer is required.

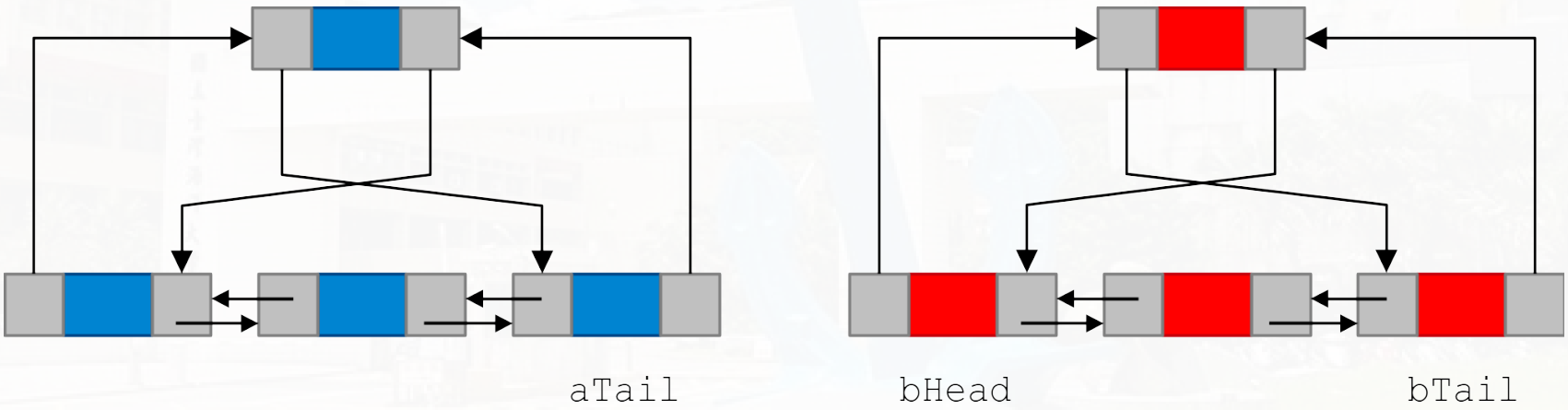


Benefit of using doubly linked circular list

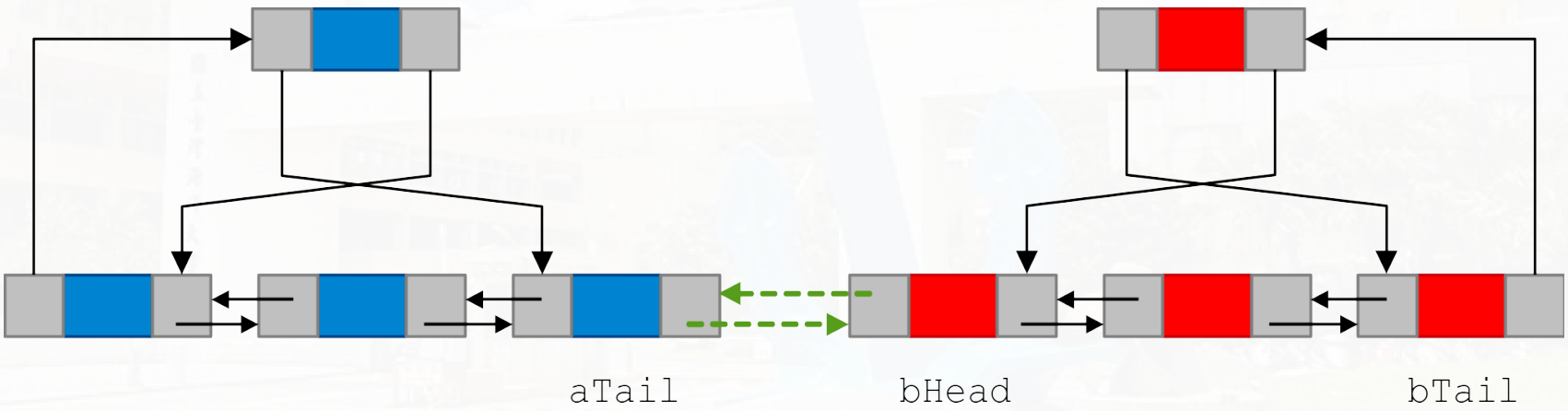
- Concatenation of two lists: $O(1)$ time.
  - Using the header node.
- Inversion of a list: $O(1)$ time (consider `llink` as `rlink` and vice versa).



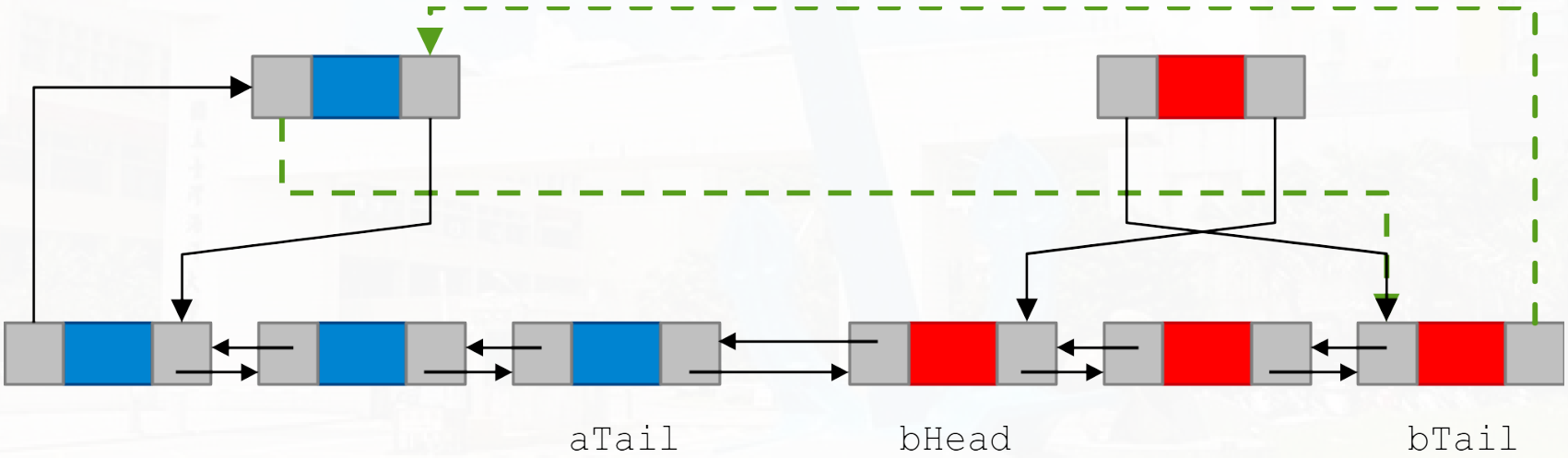

Concatenation of two doubly linked circular list



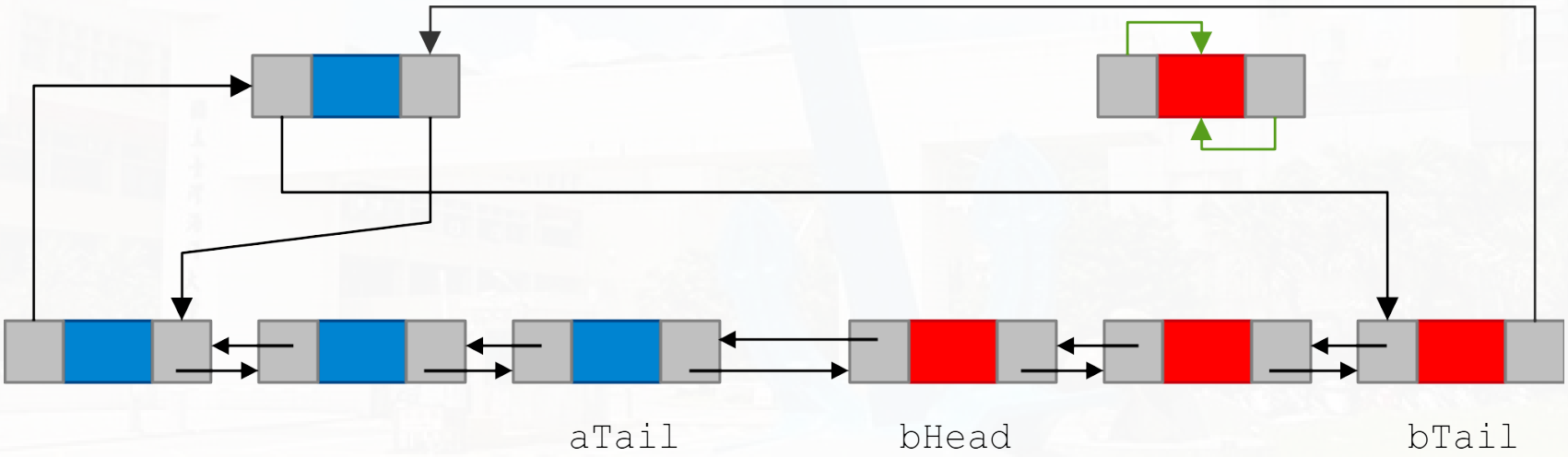
Concatenation of two doubly linked circular list



Concatenation of two doubly linked circular list



Concatenation of two doubly linked circular list



Concatenation of two doubly linked circular lists

```
void concat(Node *headerA, Node *headerB) {
    if (headerA == headerB) return;  /* avoid self-append */
    if (headerB->rlink == headerB) return; /* B is empty so skipped*/

    Node *aTail = headerA->llink;   // get the last node of A
    Node *bHead = headerB->rlink;   // get the first node of B
    Node *bTail = headerB->llink;   // get the last node of B

    // Stitch B after A's tail
    aTail->rlink = bHead;
    bHead->llink = aTail;

    // Close the ring back to headerA
    bTail->rlink = headerA;
    headerA->llink = bTail;

    // Make B empty
    headerB->rlink = headerB->llink = headerB;
}
```




Discussions

